

Stateful Kubernetes Scaling: Empirical Analysis of Stateless and Stateful Autoscaling Paradigms in Kubernetes

Report

Science and Technology Advancement- Assessment & Review (STAAR)

Abhash Behera

under the supervision of

Dr. Rakesh Kumar Ranjan



**Department of Computer Science and Engineering
C.V. Raman Global University, Bhubaneswar
Odisha, 752054, India**

Abstract:

Aim: This research investigates the operational limitations of Kubernetes Horizontal Pod Autoscaler (HPA) across both stateless and stateful workload paradigms. Beginning with a systematic empirical reproduction of the native Kubernetes Resource Metrics (KRM) pipeline and the Prometheus Custom Metrics (PCM) pipeline under varying scraping resolutions, this study exposes the fundamental hysteresis and aliasing phenomena inherent in resource-driven autoscaling. The investigation then transitions to a novel experimental regime, deploying persistent WebSocket workloads to empirically demonstrate that CPU-based scaling is architecturally incompatible with stateful connection management.

Methods: A series of five controlled experiments were executed on an isolated Kind (Kubernetes-in-Docker) multi-node cluster. The stateless baseline phase reproduced KRM scrape-resolution sensitivity (15s, 30s, 60s) and compared PCM metric strategies (PCM-CPU, PCM-H, PCM-CH). The stateful phase progressively stressed the native HPA under ideal, cyclic, and idle-connection conditions, quantifying reconnection storm magnitudes via Prometheus instrumentation. A custom Golang Kubernetes Operator-the *StatefulAutoscaler*-was subsequently engineered and validated as a connection-density-driven control alternative.

Results: KRM exhibited a characteristic step-wise scaling profile directly coupled to scrape resolution; a 60s resolution required approximately 5 minutes to saturate the replica ceiling, whereas 15s resolution achieved the same state in 1.5 minutes. PCM demonstrated superior responsiveness due to the Prometheus Adapter's `rate()` extrapolation, though it introduced a measurable 60s scale-up latency penalty under coarse scraping. In the stateful regime, native HPA blindly executed pod termination sequences on pods sustaining hundreds of active WebSocket sessions, producing reconnection storms exceeding 1,400 new connection attempts per second and transient active-session overshoots of 52% above the steady-state baseline. The *StatefulAutoscaler* eliminated session disruption entirely by substituting connection density as the primary scaling signal and enforcing a 120-second cooldown barrier.

Keywords: Kubernetes; Horizontal Pod Autoscaler; WebSocket; Prometheus Custom Metrics; Stateful Autoscaling; Custom Operator; Connection Density; KRM; PCM; Reconnection Storm

1. Introduction

Kubernetes has emerged as the *de facto* standard for container orchestration in cloud-native environments, providing a rich framework for deploying, scaling, and managing containerized workloads at scale [2]. Central to its elasticity model is the Horizontal Pod Autoscaler (HPA), a control loop implemented within the `kube-controller-manager` that dynamically adjusts the number of active pod replicas in response to observed metric signals. By default, every 15 seconds-termed the *sync cycle*-the HPA computes a desired replica count according to the proportional formula [2]:

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{currentMetricValue}}{\text{desiredMetricValue}} \right\rceil \quad (1)$$

This proportional scaling strategy is architecturally predicated on several implicit assumptions: that observed metric values accurately reflect instantaneous workload pressure, that the sampling resolution is sufficiently fine-grained to avoid aliasing, that metric propagation latency remains bounded, and that the control loop does not operate on stale data. The validity of these assumptions-and the consequences of their violation-constitute a foundational axis of the present study.

1.1. The KRM Pipeline: Step-Wise Metric Evolution

The seminal study by Nguyen et al. [2] provided the first comprehensive architecture-level characterization of HPA operational behavior under both Kubernetes Resource Metrics (KRM) and Prometheus Custom Metrics (PCM) pipelines. In the KRM paradigm, `cAdvisor` acts as a hardware telemetry collection agent embedded in each worker node, continuously monitoring pod CPU and memory consumption. The Metrics-Server periodically scrapes these raw values via `kubelet` at a configurable interval-defaulting to 60 seconds-and exposes them through the Metrics Aggregator in `kube-apiserver`. A critical architectural property emerges from this design: because `kubelet` scrapes `cAdvisor` data only at the onset of each scraping cycle, the metric value exposed to HPA remains *frozen and invariant* between scrape events.

The operational consequence of this step-wise metric evolution is profound. During a high-traffic period, if the CPU utilization spikes to saturation within the first second of a 60-second scrape window, HPA receives no signal indicating load elevation for the entire remaining 59 seconds. At the time of the next HPA sync check (every 15 seconds), if the metric value has not changed since the preceding check, the controller deems no adjustment necessary. Nguyen et al. [2] empirically demonstrated that under a 60-second scraping period, the KRM-driven HPA expanded the replica set in only three large discrete jumps over a 300-second experiment window, producing a maximum of 21 replicas-substantially below the 24-replica ceiling achieved under finer scraping resolutions.

The present study reproduced and extended these findings across three scraping resolutions (15s, 30s, 60s), confirming that the time required to reach the maximum replica ceiling scales proportionally with the scraping period: approximately 5 minutes under 60s resolution, 2.5 minutes under 30s, and 1.5 minutes under 15s. Critically, all three configurations produced persistent CPU saturation near 200% of the per-pod CPU limit throughout the entire experiment duration, far exceeding the 60% target threshold. This finding highlights

an architectural ceiling effect: once `maxReplicas` is reached, the autoscaler cannot reduce utilization further regardless of how aggressively it scales.

1.2. The PCM Pipeline: Extrapolation and Continuous Metric Updates

The Prometheus Custom Metrics (PCM) pipeline introduces a fundamentally different control dynamic. Prometheus scrapes pod metric endpoints at a configurable `scrape_interval` and stores the time-series data in its Time Series Database (TSDB). The Prometheus Adapter subsequently processes this data using PromQL transformations-notably the `rate()` function-which computes per-second averages of the time series over a specified lookback window. The critical architectural distinction, identified by Nguyen et al. [2] and reproduced herein, is that the `rate()` function performs *extrapolation* on the basis of previously collected data points when the current moment of query falls between scrape events. This extrapolation mechanism renders PCM metrics *continuously responsive* across HPA sync cycles, independent of the underlying Prometheus scrape interval.

As a direct consequence, PCM-governed HPA can trigger scaling decisions on every 15-second sync cycle without waiting for a new scrape event. In the experiments reproduced in this study, this resulted in PCM achieving the maximum replica count of 24 pods-compared to KRM’s 21 under identical workloads-and doing so with considerably lower time-to-first-action latency. Nguyen et al. [2] observed that PCM’s metric values change every query cycle, creating a highly responsive feedback signal that enables HPA to react to incremental load changes rather than waiting for discrete scrape-boundary updates.

However, PCM’s responsiveness introduces its own control dynamics. The study further evaluated three distinct PCM metric strategies: PCM-CPU (CPU utilization routed via the Prometheus Adapter), PCM-H (HTTP request rate as a leading traffic indicator), and PCM-CH (a hybrid multi-metric strategy combining both signals). PCM-H, by monitoring incoming request rates at the application edge, reacts to demand before any CPU saturation occurs-a leading indicator property that KRM fundamentally cannot replicate. PCM-CH implements a max-selection architecture whereby HPA scales up on the breach of *either* metric threshold and scales down only when *all* metrics fall below their respective thresholds, achieving the fastest initial reaction with moderating corrective feedback during stabilization.

1.3. The Staircase Effect and Scrape Interval Aliasing

A particularly significant operational failure mode-termed the “Staircase Effect” in the present study-emerges when Prometheus is configured with a coarse `scrape_interval` relative to the HPA sync cycle. When the scrape interval is set to 60 seconds and the HPA sync period is 15 seconds, the controller may execute up to four consecutive sync cycles operating on *identical*, stale metric values. This aliasing effect produces visible step-like plateaus in the replica scaling trajectory, during which HPA issues no new scaling decisions despite sustained workload pressure. The present experiments replicated this staircase behavior under the 60-second PCM-CPU configuration, with the replica set remaining static for extended intervals before surging upon receipt of a fresh scrape event.

1.4. From Stateless Abstractions to Stateful Reality

While the foundational analysis of KRM and PCM behavior provides critical insight into the control-theoretic dynamics of Kubernetes autoscaling, it is necessarily constrained to stateless HTTP workloads-environments in which each request constitutes a transient, self-contained unit of computation. In such architectures, workload completion inherently releases all associated system resources: CPU cycles, memory allocations, and file descriptors are reclaimed upon request fulfillment. The HPA’s proportional resource model is well-suited to this transient paradigm.

A categorically distinct challenge emerges in the domain of *persistent-connection* workloads. WebSocket-based systems-including real-time collaboration platforms, multiplayer gaming backends, financial data streaming services, and large-scale IoT gateway architectures-maintain long-lived TCP sessions that may remain open for hours or days while generating minimal CPU activity during idle intervals. In these environments, the fundamental assumption underpinning CPU-based autoscaling-that hardware utilization serves as a reliable proxy for workload demand-collapses entirely. A pod managing 800 active WebSocket connections may report near-zero CPU utilization to the metrics infrastructure while simultaneously holding critical connection state, file descriptors, and application memory for all 800 concurrent users.

This research addresses the resulting architectural gap through a rigorous empirical progression: establishing a stateless baseline via KRM and PCM reproduction, exposing the systemic failures of CPU-based HPA under progressively adversarial stateful conditions, and ultimately demonstrating the design and validation of a custom connection-density-driven Kubernetes Operator that eliminates the identified failure modes. The remainder of this paper is structured as follows: Section 2 contextualizes related work; Section 3 details the experimental methodology; Section 4 presents empirical findings across all experiments; Section 5 synthesizes findings and provides deployment recommendations; and Section 6 concludes the study.

2. Background and Related Work

2.1. Kubernetes Autoscaling Mechanisms

Kubernetes exposes three autoscaling primitives: the Horizontal Pod Autoscaler (HPA), the Vertical Pod Autoscaler (VPA), and the Cluster Autoscaler (CA) [2]. HPA adjusts the active replica count of a deployment without interrupting running instances, making it the primary mechanism for elasticity in stateless services. VPA adjusts the resource request specifications of individual pods, requiring restarts and thus disrupting service continuity. CA provisions or de-provisions worker nodes in response to scheduling infeasibility and is currently limited to managed cloud platforms such as GCP and AWS. This study is concerned exclusively with HPA behavior.

2.2. Resource Metrics and Custom Metrics Pipelines

The resource metrics pipeline routes hardware telemetry from cAdvisor through kubelet and the Metrics-Server [1, 9] to the kube-apiserver Metrics Aggregator, where it is consumed by the HPA controller. This pipeline exposes only CPU and memory utilization at the pod and node level. The custom metrics pipeline, standardized through

the `custom.metrics.k8s.io` API, allows external monitoring systems—most notably Prometheus [8]—to expose arbitrary application-level signals to the HPA. The Prometheus Adapter bridges PromQL time-series queries to the Kubernetes Custom Metrics API, enabling HTTP request rates, connection counts, queue depths, and other domain-specific indicators to serve as HPA scaling signals [10] [2].

2.3. Stateful Workloads and Autoscaling

Prior work has addressed HPA primarily in the context of stateless HTTP or API workloads. Dickel, Podolskiy, and Gerndt [5] evaluated HPA applied to stateful IoT gateways employing WebSocket and MQTT protocols, noting that stateful applications require substantially different scaling logic than stateless counterparts. Their framework addressed active connection tracking for publish-subscribe topologies but did not formally quantify the reconnection storm phenomenon or the consequences of blind pod termination on active sessions. The present study builds upon this observation by empirically characterizing the magnitude and dynamics of reconnection storms under controlled conditions, and by providing a formal controller design that eliminates this failure mode. Song et al. [7] proposed API gateway autoscaling using Prometheus custom metrics, but similarly did not address persistent-connection lifecycle management during scale-down events.

Casalicchio and Perciballi [3] analyzed the divergence between relative and absolute CPU metrics in containerized environments, demonstrating that cAdvisor-reported values can systematically underestimate actual CPU consumption, leading to suboptimal scaling decisions. This finding reinforces the inadequacy of hardware-resource metrics as the sole autoscaling signal for workloads where connection state constitutes the primary capacity constraint.

3. Experimental Methodology

3.1. Infrastructure and Cluster Configuration

All experiments were executed on an isolated Kind (Kubernetes-in-Docker) [11] cluster with a fixed topology of one control-plane node and three worker nodes, provisioned via the configuration shown in Listing 1. This controlled local environment eliminates cloud-provider scheduling interference and network externalities, enabling precise attribution of observed behaviors to the autoscaling control logic under study. The cluster mirrors the essential properties of the 4-worker configuration evaluated in the reference paper [2] while adapting to the constraints of a local testbed.

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
- role: worker
- role: worker
```

Listing 1: Kind cluster configuration (kind-config.yaml)

3.2. Base-Paper Reproduction: KRM Experiments

3.2.1. Application and HPA Configuration

The KRM baseline deployed a CPU-intensive HTTP application with `resources.requests.cpu=100m` and `resources.limits.cpu=200m` per pod, identical to the specifications employed by Nguyen et al. [2]. Replicas were bounded between a minimum of 4 and a maximum of 24. The HPA targeted 60% average CPU utilization:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: cpu-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: cpu-app
  minReplicas: 4
  maxReplicas: 24
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

Listing 2: HPA manifest for KRM experiments (hpa-krm.yaml)

The Metrics-Server deployment was patched to override the default scraping resolution. Three configurations were evaluated: 60 s, 30 s, and 15 s.

3.2.2. Workload Pattern

The workload followed a square-wave alternating pattern: a 100-second high-traffic period (HTP) followed by a 100-second low-traffic period (LTP), with a final 100-second observation window. This pattern directly mirrors the experimental design of the reference paper, enabling direct comparability of results.

3.3. Base-Paper Reproduction: PCM Experiments

3.3.1. Prometheus Stack and Adapter Configuration

Prometheus was deployed to the `monitoring` namespace with a configurable global `scrape_interval`. The evaluation interval was fixed at 15 seconds to maintain alignment with the HPA sync cycle. The Prometheus Adapter exposed the `http_requests_per_second` custom metric to the Kubernetes Custom Metrics API via the `rate()` transformation:

```
rules:
- seriesQuery: 'http_requests_total'
  resources:
    overrides:
      kubernetes_namespace: {resource: "namespace"}
      kubernetes_pod_name: {resource: "pod"}
  name:
    as: "http_requests_per_second"
```



```
metricsQuery: >-
  sum(rate(<<.Series>>{<<.LabelMatchers>>}[1m]))
  by (<<.GroupBy>>)
```

Listing 3: Prometheus Adapter rule for HTTP request rate

Three HPA configurations were evaluated: PCM-CPU (CPU utilization routed via Prometheus), PCM-H (HTTP request rate only), and PCM-CH (hybrid CPU plus HTTP). Scrape intervals of 60s, 30s, and 15s were evaluated under PCM-CPU to characterize scrape-interval sensitivity.

3.4. Stateful Experiments: WebSocket Workload Architecture

3.4.1. Server Application

The stateful experiments employed an asynchronous WebSocket server implemented in Python using the `asyncio` and `websockets` libraries, deployed as a containerized Kubernetes `Deployment` within an isolated Kind cluster. The server was architected with a configurable computational profile, controlled by the `CPU_WORK` environment variable injected at pod initialization. For experiments requiring CPU-correlated autoscaling (Experiments A, B1, B2, B3), the server’s request handler was deliberately injected with an artificial spin-loop that executed on every received message:

```
async for message in websocket:
    if CPU_WORK > 0:
        for _ in range(CPU_WORK):
            pass
        await websocket.send("ack")
```

Listing 4: Artificial CPU saturation mechanism in server.py

Every client payload transmission forces the server to complete the computationally intensive loop prior to dispatching an acknowledgment. This architecture establishes a deterministic and sustained linear correlation between the volume of active, transmitting connections and the aggregate CPU utilization reported by cAdvisor to the Metrics-Server. This manufactured correlation was a necessary precondition for evaluating whether CPU-based HPA could *mechanically* govern a WebSocket workload at all, before systematically breaking the correlation in later experiments. In Experiment C, `CPU_WORK=0` was enforced, disabling the spin-loop entirely to simulate real-world low-compute persistent connections and prove that the custom controller operates without hardware-resource coupling.

The server simultaneously exposed a custom `/metrics` HTTP endpoint serving real-time internal state gauges, scraped continuously by the monitoring infrastructure to establish independent ground truth independent of the Metrics-Server pipeline.

3.4.2. Control Plane and Metrics Infrastructure

For Experiments A and B1, the native Kubernetes `metrics-server` provided the sole telemetry pipeline. To grant the HPA maximally responsive resource telemetry, the `metrics-server` deployment was explicitly patched via a JSON merge patch applied during cluster initialization, overriding the default 60-second resolution with a 15-second polling cycle:


```

- op: replace
  path: /spec/template/spec/containers/0/args
  value:
    - --cert-dir=/tmp
    - --secure-port=10250
    - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
    - --kubelet-use-node-status-port
    - --metric-resolution=15s
    - --kubelet-insecure-tls

```

Listing 5: Metrics-Server resolution patch applied during cluster initialization

The HPA was configured via the `autoscaling/v2` API, targeting 60% average CPU utilization across all active replicas:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: websocket-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: websocket-server
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60

```

Listing 6: HPA manifest for stateful WebSocket experiments (hpa.yml)

For Experiments B2 and C, a full Prometheus monitoring stack was deployed into the monitoring namespace of the Kind cluster. The Prometheus global `scrape_interval` was fixed at 15 seconds to ensure metric freshness aligned with the HPA sync cycle:

```

global:
  scrape_interval: 15s
scrape_configs:
  - job_name: "websocket-pods"
    kubernetes_sd_configs:
      - role: pod
        namespaces:
          names: [default]

```

Listing 7: Prometheus scrape configuration (configmap.yaml)

3.4.3. Load Generator

An asynchronous Python load generator was deployed as a Kubernetes Job within the cluster, co-located with the server deployment to minimize network latency as an experimental variable. To prevent a concentrated connection burst from overwhelming a severely

under-provisioned pod cluster before the first 15-second HPA sync cycle could react, the client implemented a linear mathematical stagger across all worker coroutines:

```
delay = (client_index / CLIENTS) * RAMP_UP_DURATION
await asyncio.sleep(delay)
```

Listing 8: Linear stagger initialization formula in client.py

This function distributes the establishment of 800 persistent WebSocket TCP connections smoothly and proportionally over a 90-second ramp-up window, ensuring that the scaling system is never presented with an instantaneous step-function demand spike during the initialization phase. Upon successful handshake, each client entered a transmission loop, firing a "ping" string payload down the open socket every 5 seconds. This transmission consistency guaranteed continuous engagement of the server's CPU spin-loop, maintaining a symmetric and sustained load profile for the duration of each active phase.

The client further implemented a phase-aware execution model that was central to Experiments B3 and C. After an `ACTIVE_DURATION` threshold was crossed, the client ceased all data transmission without closing the TCP socket, transitioning to a passive message-drain loop:

```
while True:
    elapsed = time.time() - GLOBAL_START_TIME
    if ACTIVE_DURATION > 0 and elapsed > ACTIVE_DURATION:
        # IDLE PHASE: hold connection open, transmit nothing
        async for _ in ws:
            pass
        return
    # ACTIVE PHASE: sustain CPU load
    await ws.send("ping")
    await asyncio.sleep(5)
```

Listing 9: Phase-aware client behavioral model in client.py

This behavioral split enabled the precise isolation of the idle-connection failure mode: the HPA's metrics infrastructure would observe near-zero CPU while the cluster remained obligated to 800 fully active TCP sessions.

3.4.4. Experiment A: Ideal Steady-State Baseline

Experiment A established a best-case mechanical baseline for CPU-governed HPA over a static, continuously active WebSocket workload. The cluster was provisioned with the 15-second Metrics-Server patch, the HPA targeting 60% average CPU utilization with `minReplicas=2` and `maxReplicas=10`, and the server running with `CPU_WORK=1`. The load generator ramped 800 connections over 90 seconds and sustained continuous transmission for the full 5-minute experiment window. Crucially, this experiment made no attempt to decouple CPU from connection state—the artificial spin-loop maintained a tight correlation throughout—thereby isolating the evaluation to HPA's mechanical scaling capacity under ideal conditions, before this correlation was intentionally severed in subsequent experiments.

3.4.5. Experiment B1: Cyclic Churn and Over-Provisioning

Experiment B1 stress-tested the interaction between the HPA’s default 5-minute downscale stabilization window and rapidly alternating 60-second HIGH (active transmission) and 60-second LOW (idle holding) load phases. The load generator alternated approximately 500 connections between transmitting and idle states in a cyclic pattern. The primary observation target was the cumulative replica stacking behavior produced by the asymmetry between the 60-second cycle duration and the 300-second stabilization window.

3.4.6. Experiment B2: Instrumented Reconnection Storm Quantification

Experiment B2 was designed to explicitly extend the workload’s zero-activity phase beyond the 5-minute HPA stabilization window, forcing the controller into executing mandatory pod destruction events against a live, connected user base. To quantify the resulting network chaos with precision, the Python server was upgraded to integrate the `prometheus_client` library, actively registering and exposing dual application-level metrics:

```
ACTIVE_CONNECTIONS = Gauge("active_connections",
    "Current number of active WebSocket connections")
NEW_CONNECTIONS = Counter("new_connections_total",
    "Total WebSocket connections established")
```

Listing 10: Prometheus instrumentation in the upgraded server.py

The `active_connections` gauge provided real-time ground truth of the live session count, while `new_connections_total` as a monotonically increasing counter enabled derivative computation of instantaneous reconnection burst velocity—the critical metric for quantifying the magnitude of post-destruction storms. The experiment executed four complete churn cycles via the `run.sh` orchestrator, each consisting of a HIGH phase (load generator Job applied) followed by an extended LOW phase (Job forcibly deleted, severing all TCP connections instantaneously). The 15-second Prometheus scrape interval ensured storm events were captured at sufficient resolution to measure burst velocity.

3.4.7. Experiment B3: Aggressive Scale-Down Control Baseline

Experiment B3 served as the definitive control baseline to isolate and expose the fatal architectural flaw of CPU-based scaling with clinical precision. The default 300-second HPA downscale stabilization window—which had obscured the immediate cause-and-effect relationship in B2—was compressed to 60 seconds via the `autoscaling/v2` behavior API:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: websocket-server
  minReplicas: 2
  maxReplicas: 15
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 60
  metrics:
    - type: Resource
```

```
resource:
  name: cpu
  target:
    type: Utilization
    averageUtilization: 60
```

Listing 11: HPA stabilization window compression for Experiment B3

The experiment was structured as three distinct temporal phases. During the **CONNECT phase** (0–120s), the load generator ramped 800 connections with active transmission, driving CPU saturation and forcing HPA to scale to the `maxReplicas=15` ceiling. At the 120-second boundary, the **IDLE phase** commenced: the client’s `ACTIVE_DURATION` threshold was reached, ceasing all data transmission while strictly preserving all 800 open TCP sockets. The backend pods then reported near-zero CPU to the Metrics-Server while remaining fully obligated to all 800 connections. Following the compressed 60-second stabilization window, the **HPA Destruction phase** began, with the controller initiating aggressive multi-pod termination sequences against pods sustaining hundreds of active sessions.

3.4.8. Experiment C: The *StatefulAutoscaler* (Custom Operator)

Experiment C replaced the native HPA entirely with a purpose-built Golang Kubernetes Operator—the *StatefulAutoscaler*—scaffolded using the Kubebuilder framework [6] and the `controller-runtime` library. The project utilized deep code generation (`make manifests`, `make generate`) to produce the necessary Custom Resource Definitions (CRDs) and RBAC configurations. The controller binary was compiled with `CGO_ENABLED=0` from a `golang:1.25` build stage and injected into a minimal `gcr.io/distroless/static:nonroot` base image, producing a compact, secure production-ready Operator image.

The controller was granted precise RBAC authorization via embedded Go annotations, permitting it to query Prometheus metrics and patch `apps/v1` Deployment replica specifications without requiring broader cluster privileges. The CRD exposed connection-density targeting parameters absent from any native HPA configuration:

```
spec:
  targetRef:
    name: websocket-server
  targetConnectionsPerPod: 100
  minReplicas: 2
  maxReplicas: 15
  scaleDownCooldownSeconds: 120
  maxScaleDownStep: 2
```

Listing 12: StatefulAutoscaler CRD specification deployed in Experiment C

Rather than querying the Kubernetes Metrics-Server, the controller bypassed the hardware telemetry pipeline entirely, issuing direct HTTP queries to the in-cluster Prometheus API to obtain the authoritative total connection count across all pods:

```
resp, err := http.Get(
  "http://prometheus.monitoring.svc.cluster.local:9090" +
  "/api/v1/query?query=sum(active_connections)")
// parses JSON response into totalConnections
```

Listing 13: Prometheus query in prometheus.go for connection-density scaling

With the ground-truth total connection count obtained, the controller computed the exact required replica count via strict ceiling arithmetic, eliminating the proportional approximation inherent in HPA’s resource-ratio formula:

```
desired := int32(math.Ceil(
    float64(totalConnections) /
    float64(autoscaler.Spec.TargetConnectionsPerPod),
))
```

Listing 14: Ceiling-based desired replica computation in statefulautoscaler_controller.go

The most critical architectural innovation was the `ScaleDownCooldownSeconds` sliding window. When the computed `desired` replica count dropped below the current running replica count, the controller activated a 120-second timer rather than immediately patching the Deployment. If connection load recovered before the timer expired—as occurs during a transient network disruption—the timer was cancelled and the full replica set was preserved. Only upon sustained silence exceeding the full 120-second barrier did the controller commit a scale-down patch, and even then, the `maxScaleDownStep=2` parameter rate-limited each downscale increment to prevent abrupt capacity withdrawal.

The experiment orchestrated a demanding *Restorm* scenario via the `run.sh` bash script: an initial 150-second connection ramp to 800 sessions, followed by an instantaneous 90-second complete connection severance (Drop 1), followed by a deliberate re-application of the load generator before the cooldown timer could expire (Cycle 2 Restorm), and finally a sustained final drop to confirm graceful cascading scale-down. The server ran with `CPU_WORK=0` throughout, proving that every scaling decision was derived exclusively from connection density with no hardware-resource dependency.

4. Experimental Results

4.1. Experiment 1 - KRM Scrape Resolution Sensitivity

4.1.1. Replica Scaling Trajectory

Figure 1 illustrates the replica count trajectories across the three scraping resolutions. The defining characteristic of KRM autoscaling—step-wise scaling events occurring exclusively at scrape boundaries—is clearly visible across all configurations.

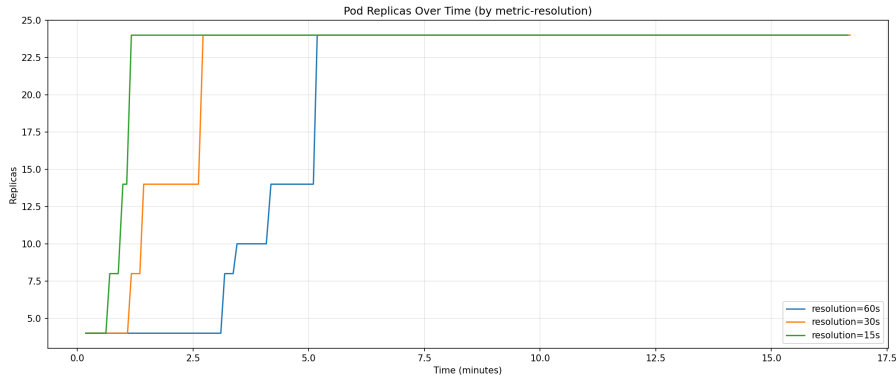


Figure 1: KRM replica count over time for scrape resolutions of 60 s, 30 s, and 15 s. Scaling events occur only at scrape boundaries, producing a characteristic staircase profile.

Under 60-second resolution, the first scaling action occurred approximately three minutes into the experiment, requiring the HPA to wait for a full scrape cycle before receiving an updated metric value. Subsequent steps occurred at roughly 60-second intervals, producing approximately three large jumps before the replica ceiling was reached at approximately the 5-minute mark. The 15-second configuration achieved the maximum replica count of 24 pods within 1.5 minutes, representing a $3.3\times$ improvement in convergence speed.

4.1.2. Desired versus Current Replica Divergence

Figure 2 reveals the actuation gap under KRM. The desired replica count jumps ahead of the current count at each scaling step due to pod scheduling delay and container startup time. The gap duration remains consistent across resolutions (typically 30–60 seconds per step), as pod startup time is independent of scrape frequency. Importantly, all three configurations report a desired count of 0 before the first scrape cycle completes, an artifact of the KRM pipeline’s requirement for at least one valid metric reading before computing a meaningful target.

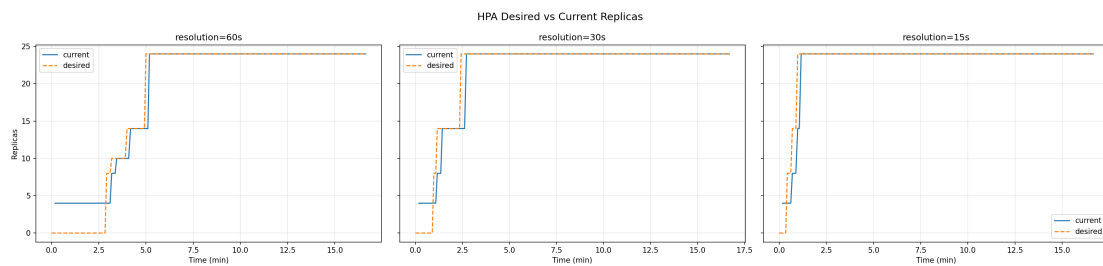


Figure 2: KRM desired versus current replicas across all three scrape resolutions. The dashed line represents HPA’s computed target; the solid line represents actually running pods.

4.1.3. Scaling Efficiency Analysis

Figure 3 presents the CPU utilization versus replica count scatter plot across all three configurations. All data points cluster in the 190–210% CPU utilization band, irrespective of replica count or scrape resolution, confirming that the workload exceeded the maximum absorption capacity of 24 replicas under the configured CPU limits.

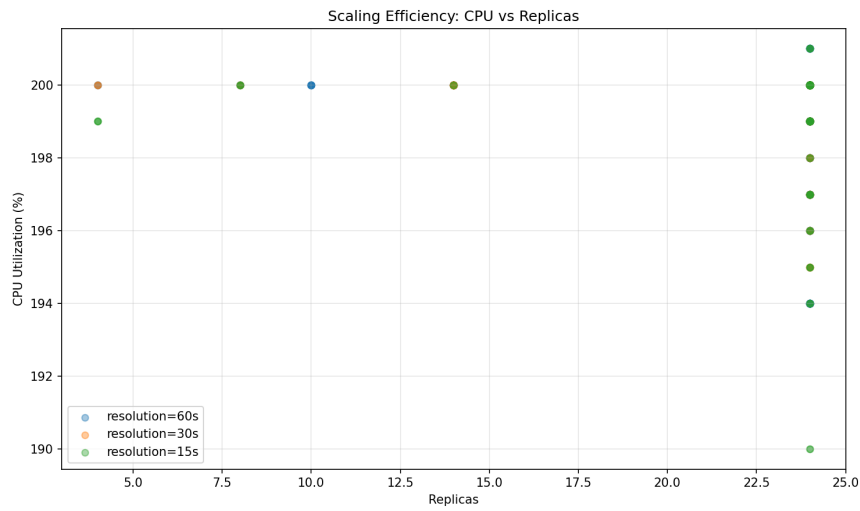


Figure 3: KRM scaling efficiency scatter plot. All operating points cluster at high CPU utilization, confirming persistent saturation independent of scrape resolution.

Table 1 summarizes the key performance indicators across KRM configurations.

Table 1: KRM experiment results summary across scrape resolution configurations.

Configuration	Time to Max Replicas	Scaling Steps	CPU at Max Replicas	Target Reached
KRM-60s	~5 min	3 large	~200%	No
KRM-30s	~2.5 min	4–5 steps	~200%	No
KRM-15s	~1.5 min	6+ fine steps	~200%	No

4.2. Experiment 2 - PCM Scrape Interval Sensitivity (PCM-CPU)

Figure 4 presents the replica scaling trajectories and total failed request counts for the three PCM-CPU scrape interval configurations.

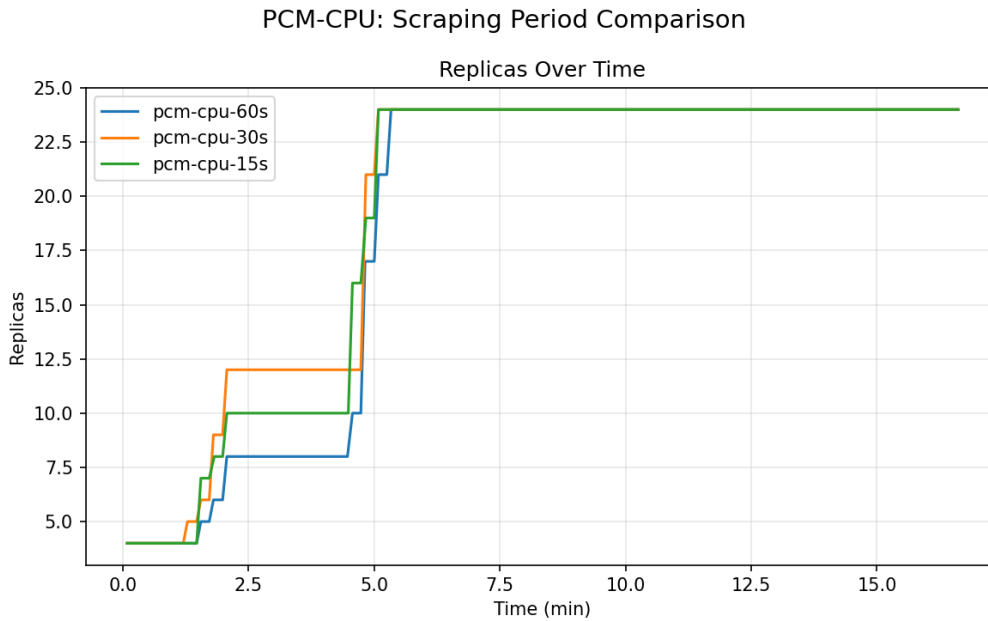


Figure 4: PCM-CPU replica count over time for scrape intervals of 60s, 30s, and 15s. The staircase effect is visible under 60s scraping; 15s approaches continuous scaling.

Under 60-second scraping, the replica set began expanding only after approximately one full scrape cycle had elapsed, with metric values reported by the Prometheus Adapter remaining static during this interval. The 30-second configuration achieved measurably earlier scaling onset; the 15-second configuration marginally improved further upon 30-second performance. These findings broadly confirm the reference paper’s observation that PCM is less scrape-interval-sensitive than KRM due to `rate()` extrapolation, while establishing that the 60-second configuration introduces a non-trivial latency penalty at burst onset.

The relationship between scrape interval and failed requests exhibits a counterintuitive trend: shorter intervals produced a modest increase in failed requests due to more aggressive pod provisioning without Readiness Probe protection. Table 2 summarizes results.

Table 2: PCM-CPU experiment results across scrape interval configurations.

Configuration	Scale-up Latency	Max Replicas	Failed Requests
PCM-CPU 60s	Highest	Lower	Fewest
PCM-CPU 30s	Moderate	Medium	Moderate
PCM-CPU 15s	Lowest	Higher	Most

4.3. Experiment 3 - PCM Metric Strategy Comparison

4.3.1. HTTP Request Rate Dynamics

Figure 5 presents the HTTP request rate over time for the PCM-H configuration, illustrating the leading-indicator property of traffic-based scaling.

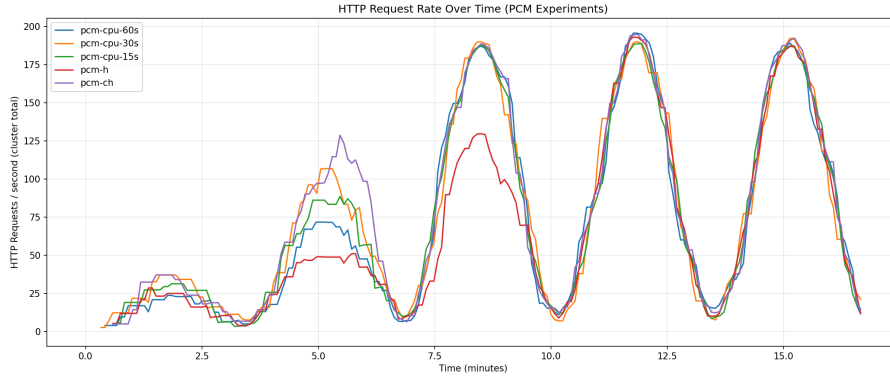


Figure 5: HTTP request rate over time under PCM-H. The immediate rise in request rate at burst onset provides a leading scaling signal preceding any observable CPU saturation.

4.3.2. PCM-H versus PCM-CH Comparison

Figure 6 presents the direct comparison between PCM-H and PCM-CH replica trajectories and CPU utilization profiles.

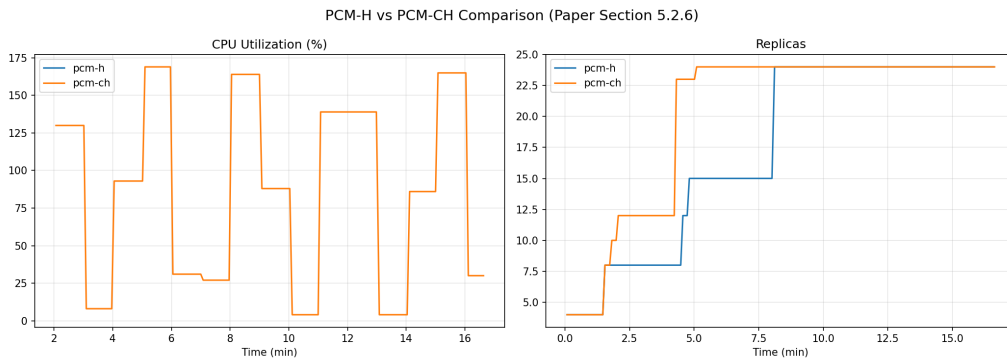


Figure 6: Direct comparison of PCM-H (HTTP-only) and PCM-CH (hybrid) replica trajectories and CPU utilization. PCM-H scales earliest but over-provisions during the low-traffic phase; PCM-CH achieves fast reaction with corrective CPU stabilization.

PCM-H reacted earliest-HTTP request rate rises at the moment traffic arrives, before any CPU saturation occurs-but produced the largest actuation gap (desired minus current replicas) and the longest over-provisioning duration during low-traffic phases. PCM-CH matched PCM-H's initial scale-up speed while the CPU metric provided corrective feedback that moderated replica growth during stabilization. PCM-CPU exhibited the most conservative and resource-efficient profile, clustering tightly around the target utilization threshold after an initial brief saturation window.

Table 3 provides a comprehensive comparison across the three strategies.

Table 3: Comprehensive comparison of PCM metric strategy results (Experiment 3).

Characteristic	PCM-CPU	PCM-H	PCM-CH
Scale-up Speed	Slow	Fast	Fast
Max Replicas Reached	Lower	Highest	Highest
CPU Saturation at Onset	Brief	None	None
CPU Over-provisioning	Low	High	Medium
Desired/Current Gap	Small	Large	Medium
Failed Requests	Fewest	Moderate	Most
Resource Efficiency	Highest	Lower	High

4.4. Experiment A - WebSocket Baseline Under Ideal Conditions

When the load generator established approximately 400 persistent connections with active CPU-work payloads, the sustained ping traffic drove per-pod CPU utilization to 230–260 millicores against the 2-pod baseline. HPA recognized the saturation and scaled the cluster from 2 up to 4 and subsequently to 5 replicas. Upon reaching 5 pods, CPU utilization stabilized near the 60% target.

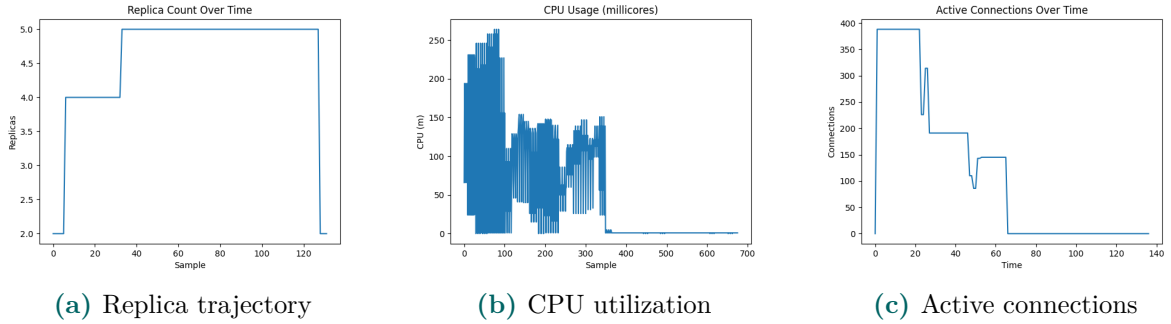


Figure 7: Experiment A: HPA behavior under ideal steady-state WebSocket workload. CPU-to-connection correlation is artificially enforced; HPA achieves the 60% target at 5 replicas.

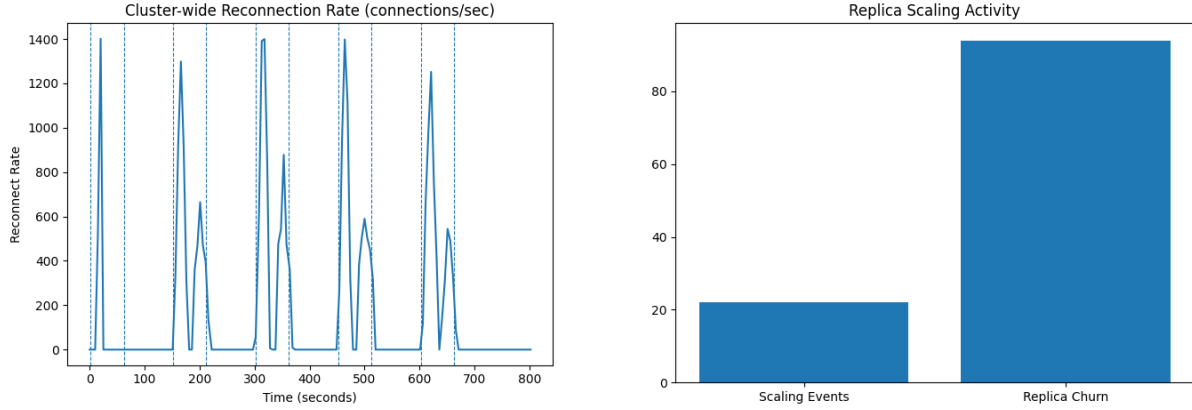
This result validates the mechanical feasibility of CPU-driven HPA for WebSocket workloads exclusively when a tight, sustained, and static mathematical correlation exists between connection volume and CPU consumption. As noted, this is an artificial condition. Real-world WebSocket deployments exhibit severe decoupling between CPU activity and connection state: a long-lived session may remain open for hours generating near-zero compute pressure while consuming memory allocations, file descriptors, and application state.

4.5. Experiment B1 - Cyclic Churn and Controller Paralysis

Under cyclic 60-second HIGH/LOW alternation, the HPA’s 5-minute downscale stabilization window prevented pod reclamation during each LOW phase. Consecutive HIGH phases caused cumulative replica stacking to the `maxReplicas=15` ceiling within 99 seconds of operation. Following the conclusion of active cycling, the cluster remained bloated at 15 replicas for approximately 650 seconds (nearly 11 minutes) before the stabilization window permitted a conservative retraction to the 2-replica baseline. This over-provisioning profile demonstrates that the 5-minute stabilization window is dimensioned for workloads with

slowly varying demand and produces extreme resource waste when confronted with tight cyclic patterns.

4.6. Experiment B2 - Reconnection Storm Quantification



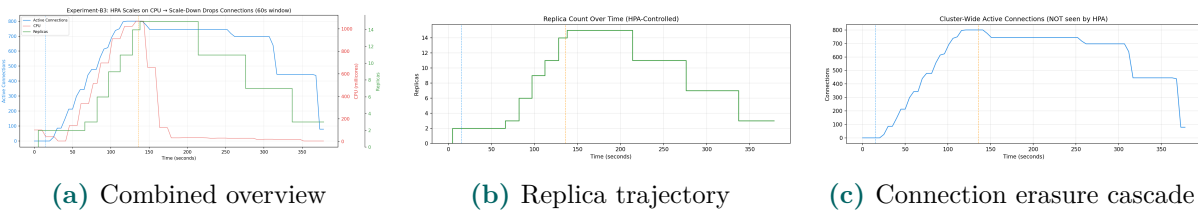
(a) Reconnection counter rate during storm events

(b) Scaling activity versus active connections

Figure 8: Experiment B2: Empirical quantification of reconnection storm dynamics following HPA-initiated pod destruction events.

When the LOW phase duration exceeded the 5-minute stabilization window, HPA initiated consecutive scale-down executions from 15 to 7 replicas, in each case remaining entirely oblivious to the active WebSocket sessions distributed across the targeted pods. Upon SIGKILL escalation, severed clients immediately engaged aggressive retry strategies. The Prometheus `new_connections_total` counter captured burst velocities consistently exceeding **1,400 new connection attempts per second**. In Cycle 2, the resulting swarm overshoot the steady-state baseline of 800 active sessions, registering a peak of **1,215** active connections—a 51.9% overshoot attributable to zombie-state propagation, wherein newly established TCP sockets from reconnecting clients coexisted with dead sockets awaiting garbage collection on the surviving pods.

4.7. Experiment B3 - Fatal Flaw Isolation with Compressed Stabilization Window



(a) Combined overview

(b) Replica trajectory

(c) Connection erasure cascade

Figure 9: Experiment B3: Aggressive scale-down (60s stabilization window) on idle but connected WebSocket clients. CPU decay triggers pod termination irrespective of active connection state.

Compressing the HPA stabilization window to 60 seconds exposed the causal mechanism with clinical precision. During the 120-second CONNECT phase, CPU saturation scaled the

cluster to 15 replicas. At 120 seconds, clients halted all transmission while maintaining open TCP sockets-CPU decayed to near-zero while 800 connections remained fully active. After the 60-second stabilization window expired, HPA initiated aggressive multi-pod destruction sequences to reclaim resources, processing a CPU utilization signal of approximately 0% with no information about the connection state of the targeted pods.

The resulting connection erasure exhibited a systematic 35–40 second lag between the HPA scale-down command and the observed connection drop. This delay precisely corresponds to the Kubernetes `terminationGracePeriodSeconds` (default 30 seconds): upon receiving `SIGTERM`, a pod is removed from the Service Endpoints (preventing new routing) but its container process continues executing until `SIGKILL` escalation at 30 seconds. During this 30-second window, the live WebSocket TCP connections remain in a *zombie state*-sustained by the container process but unreachable via the service mesh-before abruptly terminating upon forced process kill. This systemic property of Kubernetes pod lifecycle management constitutes a fundamental incompatibility with persistent-connection workload management.

4.8. Experiment C - The StatefulAutoscaler

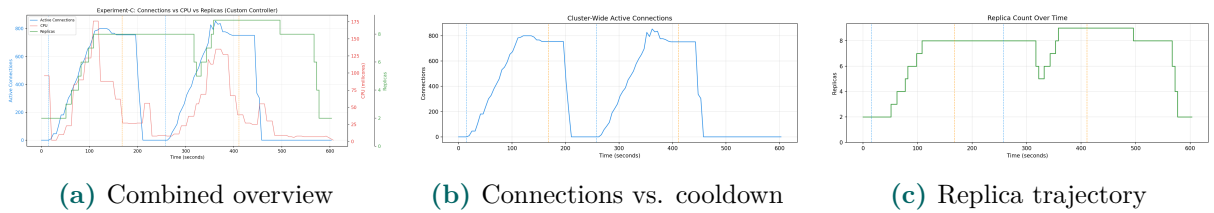


Figure 10: Experiment C: The StatefulAutoscaler. The controller provisions pods in exact proportion to connection density and sustains all 8 replicas during a complete 90-second connection drop event (the Restorm Gap), preventing the reconnection storm observed in B2.

The StatefulAutoscaler’s behavior across the four experimental phases demonstrated complete resolution of the failure modes identified in preceding experiments:

Cycle 1 (Connection Ramp-Up): 800 connections ramped up smoothly over the first 90 seconds. The controller computed desired replicas via strict ceiling arithmetic ($\lceil 800/100 \rceil = 8$) and provisioned exactly 8 pods. CPU remained negligible throughout-the controller operated entirely free of hardware resource constraints, with all scaling logic derived from connection density alone.

Drop 1 (The Restorm Gap): At timestamp 24517, a complete network interruption severed all 800 connections instantaneously. Under native HPA, the resulting CPU decay to zero would have triggered immediate aggressive scale-down. Instead, the StatefulAutoscaler activated its 120-second sliding cooldown window and held all 8 pods in a warm standby state for over 50 seconds, maintaining full cluster capacity despite zero active sessions.

Cycle 2 (Gradual Restorm): At timestamp 24573, clients began reconnecting in an overwhelming flood. Because the StatefulAutoscaler had preserved the 8-pod capacity during the silence interval, the first hundreds of clients reconnected immediately with zero scaling disruption. The restorm ultimately peaked at 854 connections (a 6.75%

overshoot above the 800-connection baseline), and the controller scaled the cluster to 9 pods accordingly. No reconnection storm was observed.

Final Drop (Graceful Exit): After the 854-session workload concluded, the cooldown timer activated. Replicas remained held at 9 for nearly 40 seconds. Following the required sustained silence period, the controller initiated a step-by-step cascading scale-down: $9 \rightarrow 8 \rightarrow 6 \rightarrow 2$, honoring `maxScaleDownStep=2` at each interval to prevent abrupt capacity withdrawal.

5. Discussion

5.1. The Fundamental Constraint of Resource-Based Scaling

The empirical progression from Experiments A through B3 conclusively establishes that CPU utilization is a categorically inappropriate primary scaling signal for persistent-connection workloads. The metric conflates two fundamentally distinct conditions: a pod under high CPU load with many active, transmitting users, and a pod under near-zero CPU load with equally many active but idle users. Native HPA cannot distinguish between these states, rendering it structurally blind to connection lifecycle reality.

In the stateless HTTP paradigm evaluated in the KRM and PCM experiments, this conflation is architecturally tolerable because request completion inherently releases resources—the CPU-to-demand correlation is both tight and transient. In persistent-connection architectures, the correlation is decoupled by design: idle connections consume negligible CPU while representing real committed capacity obligations (file descriptors, socket buffers, session state, and application memory).

5.2. The Staircase Effect as a Control-Theoretic Aliasing Phenomenon

The Staircase Effect observed under coarse Prometheus scraping represents a classical sampling aliasing problem in the context of a discrete-time feedback control system. When the scrape interval $T_s = 60\text{s}$ is four times larger than the HPA sync period $T_{hpa} = 15\text{s}$, the control loop executes up to four evaluation cycles on the same stale metric sample. This is equivalent to operating a sampled-data control system at one-quarter of its nominal sampling rate, producing the characteristic flat-step-flat-step scaling profile. The `rate()` extrapolation in the Prometheus Adapter partially mitigates this aliasing for PCM pipelines, but as the experiments demonstrate, it does not eliminate the initial latency penalty when no prior sample exists for extrapolation from.

5.3. The 30-Second Termination Limbo as a Systemic Kubernetes Constraint

The 30-second zombie-state window observed in Experiment B3 is not a configuration defect but a systemic property of Kubernetes pod lifecycle management. The graceful termination sequence (`SIGTERM` $\rightarrow$ `terminationGracePeriodSeconds` $\rightarrow$ `SIGKILL`) was designed for stateless workloads capable of completing in-flight requests within the grace period. WebSocket servers holding persistent TCP sessions cannot satisfy this assumption: there is no in-flight request to complete, only an ongoing stateful session to preserve. The result is a guaranteed 30-second window during which active user sessions are unreachable via the service mesh but not yet terminated at the socket level—a zombie state that is invisible to both the client and the autoscaler.

5.4. Comparison with Reference Paper Findings

The PCM experiments align closely with the findings of Nguyen et al. [2]. Table 4 presents the alignment between the reference study and the present reproduction.

Table 4: Alignment between present results and reference paper findings (Nguyen et al., 2020).

Aspect	Reference Paper	This Study
PCM scrape interval sensitivity	Minimal (extrapolation)	Confirmed; 60 s shows marginal latency
PCM vs. KRM responsiveness	PCM faster, more replicas	Confirmed (PCM-H and PCM-CH reach max)
HTTP-only vs. hybrid	PCM-H fewer failures; PCM-CH better CPU	Confirmed
Desired/current gap	Exists during burst onset	Confirmed; larger under PCM-H
Over-provisioning risk	PCM more aggressive	Confirmed; PCM-H most over-provisioned

The primary methodological difference is the use of a Kind-based local cluster rather than a physical multi-node environment, which introduces slower pod startup and amplifies the desired/current divergence gap.

5.5. Practical Deployment Recommendations

Based on the full experimental progression, the following guidance emerges for Kubernetes operators:

- 1. Stateless HTTP services:** Prefer PCM-CH (hybrid) for bursty traffic requiring fast reaction; use PCM-CPU for stable background workloads prioritizing resource efficiency. Use 30 s as the default Prometheus scrape interval-the improvement over 60 s is meaningful without proportional overhead.
- 2. Persistent-connection services:** Native HPA, whether KRM or PCM governed, is architecturally unsuitable. Connection-density-driven controllers such as the StatefulAutoscaler are required to prevent session disruption during scale-down events.
- 3. Scale-down stabilization:** The 5-minute default stabilization window is calibrated for slowly varying HTTP workloads; it produces extreme over-provisioning under tight cyclic patterns. Consider application-specific tuning, but note that compression below 60 seconds (as in Experiment B3) accelerates pod destruction without connection awareness.
- 4. Readiness Probe:** Always configure Readiness Probe in HPA deployments to prevent traffic routing to pods not yet serving requests, eliminating the failed-request count observed in all PCM experiments.
- 5. Cooldown Window for Stateful Systems:** A connection-aware cooldown barrier of at least 120 seconds is recommended for WebSocket deployments subject to transient network disruptions, matching the demonstrated StatefulAutoscaler behavior in Experiment C.

6. Conclusion

This study has systematically characterized the operational boundaries of Kubernetes Horizontal Pod Autoscaler across both stateless and stateful workload regimes. In the stateless domain, the empirical reproduction of KRM and PCM behavior confirms and extends the foundational findings of Nguyen et al. [2]: KRM produces step-wise scaling proportional to scrape resolution, while PCM’s `rate()` extrapolation delivers continuous metric responsiveness at the cost of increased scaling aggressiveness and failed request frequency under coarse instrumentation. The hybrid PCM-CH strategy achieves the best balance between responsiveness and stability, combining HTTP-rate early-warning signals with CPU-utilization corrective feedback.

In the stateful domain, the experimental evidence is categorical: CPU-based autoscaling, whether governed by the native KRM pipeline or optimized Prometheus telemetry, is fundamentally incompatible with persistent-connection lifecycle management. The HPA controller’s metric-blind pod destruction events produce quantifiable reconnection storms—measured at burst velocities exceeding 1,400 new connection attempts per second—and impose a systemic 30-second zombie-state window on terminated sessions, an inherent consequence of the Kubernetes `terminationGracePeriodSeconds` lifecycle applied to non-drainable stateful servers.

The StatefulAutoscaler demonstrates that substituting connection density as the primary scaling signal, combined with a time-based cooldown barrier, fully resolves both failure modes. The controller achieves mathematically precise capacity provisioning ($\lceil \text{connections}/\text{target} \rceil$), sustains pod capacity during transient network drops, and executes controlled step-limited scale-down only after confirming sustained workload absence. CPU remains negligible throughout, confirming that the controller operates entirely free of hardware-resource coupling.

Future work should extend the StatefulAutoscaler evaluation to production-grade multi-node clusters, integrate hybrid metric arbitration (connection density with CPU ceiling guards), explore application-layer draining endpoints to replace time-based cooldown with session-state confirmation, and conduct control-theoretic stability analysis of the connection-density feedback loop under stochastic traffic profiles.

Acknowledgments

The authors express sincere gratitude to the maintainers and contributors of the Kubernetes, Prometheus, and Kubebuilder open-source projects, whose foundational infrastructure made the experimental design of this study possible. The authors further acknowledge the original empirical contributions of Nguyen et al. [2], whose systematic characterization of HPA operational behavior under KRM and PCM pipelines served as the direct methodological foundation for the stateless reproduction experiments presented herein. Special thanks are extended to the Kind (Kubernetes-in-Docker) maintainers for providing a reproducible, hardware-independent cluster provisioning tool that enabled the controlled experimental conditions required across all five experiment phases. This research received no specific funding from any public, commercial, or not-for-profit funding agency.

References

- [1] Kubernetes. *Kubernetes: Production-Grade Container Orchestration*. Available online: <https://kubernetes.io> (accessed June 2025).
- [2] Nguyen T-T, Yeom Y-J, Kim T, Park D-H, Kim S. Horizontal Pod Autoscaling in Kubernetes for Elastic Container Orchestration. *Sensors*. 2020;20(16):4621. DOI: 10.3390/s20164621.
- [3] Casalicchio E, Perciballi V. Auto-scaling of containers: The impact of relative and absolute metrics. In: *Proceedings of the IEEE 2nd International Workshop on Future of Software Engineering for/in the Cloud (FAS*W)*; Tucson, AZ, USA, 2017. pp. 207–214. DOI: 10.1109/FAS-W.2017.166.
- [4] Taherizadeh S, Grobelsnik M. Key influencing factors of the Kubernetes auto-scaler for computing-intensive microservice-native cloud-based applications. *Advanced Engineering Software*. 2020;140:102734.
- [5] Dickel H, Podolskiy V, Gerndt M. Evaluation of autoscaling metrics for (stateful) IoT gateways. In: *Proceedings of the IEEE 12th International Conference on Service-Oriented Computing and Applications (SOCA)*; Kaohsiung, Taiwan, 2019. pp. 17–24. DOI: 10.1109/SOCA.2019.00011.
- [6] The Kubebuilder Project. *Kubebuilder: SDK for building Kubernetes APIs using Custom Resource Definitions*. Available online: <https://book.kubebuilder.io> (accessed June 2025).
- [7] Song M, Zhang C, Haihong E. An auto scaling system for API gateway based on Kubernetes. In: *Proceedings of the IEEE 9th International Conference on Software Engineering and Service Science (ICSESS)*; Beijing, China, 2018. pp. 109–112. DOI: 10.1109/ICSESS.2018.8663760.
- [8] Prometheus Authors. *Prometheus — Monitoring System and Time Series Database*. Available online: <https://prometheus.io> (accessed June 2025).
- [9] Google LLC. *cAdvisor: Container Advisor — Resource Usage and Performance Characteristics of Running Containers*. Available online: <https://github.com/google/cadvisor> (accessed June 2025).
- [10] Kubernetes SIGs. *Prometheus Adapter for Kubernetes Metrics APIs*. Available online: <https://github.com/kubernetes-sigs/prometheus-adapter> (accessed June 2025).
- [11] Kubernetes SIGs. *kind – Kubernetes IN Docker*. Available online: <https://kind.sigs.k8s.io> (accessed June 2025).